

1. Main Functionalities

The application displays the current date and enables users to calculate a new date by adding to or subtracting from a selected date using a specified number of days, months, or years. Users can choose a base date through a DatePicker, select a preferred date format from a ComboBox (Vietnamese: dd/MM/yyyy, US: MM/dd/yyyy, European: dd.MM.yyyy, or ISO: yyyy-MM-dd), enter a numeric value, specify whether the calculation should occur before or after the selected date, and choose the time unit (Days, Months, or Years). When the **Apply** button is pressed, the calculated date is displayed. If the input is invalid, the **Apply** button is automatically disabled and an error message is shown.

2. Design Decisions – MVC Architecture

The application is designed using the Model-View-Controller (MVC) architecture. The **DateModel** (Model) stores the selected date and display format using JavaFX properties while also providing the date calculation functionality through the `java.time.LocalDate` API. The user interface is defined declaratively in `main.fxml` and styled through `style.css` (View). The **DateController** (Controller) serves as the intermediary between the View and the Model by handling user actions and updating the Model as needed. This separation of concerns keeps the business logic independent of the user interface, resulting in a cleaner, more maintainable design.

3. Improvements Over the Previous Version

In the earlier Swing-based implementation, user interface code and application logic were combined within a single class. Event listeners were responsible for manually updating UI components, which often led to inconsistent application states. The new JavaFX version adopts the MVC pattern, eliminating this tight coupling. The Model acts as the single source of truth, while data binding ensures that the View remains synchronized automatically, removing the need for manual updates.

4. JavaFX Properties and Binding

The **DateModel** exposes the selected date through an `ObjectProperty<LocalDate>` and the chosen date format through a `StringProperty`. A `StringBinding`, created using `Bindings.createStringBinding()`, generates the formatted date string based on these properties. As a result, the displayed date updates automatically whenever the selected date or format changes. Additionally, the **Apply** button's `disableProperty()` is bound to a validation rule, ensuring that the button is enabled only when the amount field contains valid input.